

Software Design Is for Modification: The Context Minimization Principle

Leric Zhang

2026-05-17

Software design matters for exactly one reason: software must change. Yet when evaluating design choices, the industry still relies on competing heuristics rather than a shared analytical currency. This paper proposes **context cost** as that currency. We introduce the **Context Minimization Principle** (CMP): for the modifications a system must realistically support, a design is better when the sufficient context required for correct modification is cheaper to acquire.

This formulation refines a common intuition. Good design does not merely reduce how much context a modifier must understand. It also makes the remaining context discoverable, bounded, and trustworthy. CMP models this acquisition cost through two recurring forms of context expansion: **depth**, the traversal cost required to understand behavior behind a focal point, and **breadth**, the closure cost required to acquire all locations that must change together. A design move is valuable when it reduces, relocates, or indexes modification context cost more than it adds new context cost. It is over-engineered when the present context cost it imposes exceeds the future context cost it avoids.

Rather than replacing classical design principles, CMP reframes them as context operators. Boundaries reduce traversal depth by expressing responsibility contracts. Locality principles reduce breadth by co-locating or indexing change-together relationships. Architecture and patterns turn open-ended search into directed acquisition paths. Tests and language mechanisms make context checkable. Through this lens, familiar disputes between principles and over-engineering become comparable trade-offs in a single currency: the cost of acquiring sufficient context for safe modification.

1 Introduction

Software design is guided by a mature practical tradition. Information hiding isolates volatile decisions (Parnas 1972). DRY fights duplicated knowledge (Hunt and Thomas 1999). Coupling

and cohesion shape module boundaries (Yourdon and Constantine 1979). Dependency inversion protects architectural seams (Martin 2002). Clean architecture separates domain logic from infrastructure (Martin 2017). These ideas endure because they capture real engineering pressures.

The fundamental dilemma of design emerges when these principles inevitably collide. DRY advocates extracting an abstraction, yet simplicity demands local duplication (Hunt and Thomas 1999; Metz 2016). Dependency inversion protects architectural boundaries, but proliferating interfaces obfuscate execution flow. Layered architectures clarify structural responsibilities, yet force developers through a labyrinth of files before revealing concrete behavior. These conflicts are not failures of existing theory. Rather, they expose the absence of a shared currency for evaluating competing forces. When design principles point in opposite directions, how do we justify the trade-off?

This paper argues that the currency of software design is **context cost**.

The argument proceeds in three steps. First, all engineering that happens after a codebase exists is reframed as modification: feature work, debugging, refactoring, review, and testing are all operations around changing a living system. Second, correct modification requires information completeness: knowing what to change, what to preserve, and which constraints must remain true, this sufficient information set is defined as the **required context** of a modification. Third, the principle is refined: Good design minimizes the cost of acquiring **required context** for future modification.

The remainder of the paper develops this idea through two recurring shapes of context acquisition. **Depth** asks how far a modifier must traverse to understand behavior behind a focal point. **Breadth** asks how reliably the modifier can acquire the full closure of locations that must change together. With these two axes in view, abstraction, duplication, architecture, patterns, testing, programming languages, AI-assisted engineering, and over-engineering can all be analyzed as context-acquisition trade-offs.

2 Software Development as Modification

The traditional distinction between software “development” and “maintenance” is a project-management fiction, echoing software-evolution work that treats useful software as continually changed rather than simply completed (Lehman 1980). It may help with budgeting, but it obscures the essential fact of programming work: once a codebase exists, almost every subsequent engineering act is an act of modification.

Software is not produced as a single uninterrupted act of creation. It accretes through a sequence of changes to a partial system. Adding a feature modifies the current state. Fixing a bug alters behavior to match intent. Refactoring modifies structure while preserving behavior. A public release is not the boundary between creation and maintenance; it merely adds new constraints to an ongoing evolutionary process.

Through all these acts, software is continuously modified to align with expected behavior. Those expectations inevitably change (Lehman 1980). Product strategies pivot, domain rules mutate, external integrations evolve, security requirements tighten, and usage patterns reveal assumptions that were false. A living system is never designed for one isolated future. It must survive a stream of modifications whose exact direction, frequency, and shape remain uncertain.

Because change is the defining condition of useful software, modifiability is not merely one quality attribute among many. It is the problem software design exists to address. Design, as a discipline distinct from making code execute, is the practice of making future modifications simpler, safer, and more reliable. Information hiding shields likely changes. DRY prevents widespread collateral updates. Architecture predicts where changes should land. Tests reveal whether a change preserved behavior. The payoff of good design is not deferred to a mythical maintenance phase; it accelerates the very next modification.

3 Context as Information Completeness for Modification

The software industry has long championed “maintainability” and “readability” as primary design goals. Yet neither is an end in itself. We maintain and read code for one operational purpose: to modify it. A developer does not study a codebase for passive comprehension; they read to make decisions about what to alter, what to preserve, and how to safely navigate constraints. Even debugging, reviewing, and testing are not distinct disciplines, but supporting operations within this broader modification pipeline. They exist to establish whether a change can be safely made, where it belongs, and whether it was executed correctly.

To execute a modification correctly, a developer or AI agent must achieve information completeness. They cannot change a system reliably while guessing its constraints. They must acquire a sufficient set of facts to perform the change and preserve intended behavior. We call this sufficient information set the **required context** of a modification.

Context is the sum of all knowledge required to execute a change. This knowledge fundamentally takes two forms: explicit and implicit. *Explicit context* is information formally encoded in written artifacts—the source code itself, schemas, tests, and documentation. *Implicit context* is the unwritten information a developer must hold in their head—runtime behaviors, hidden global states, historical decisions, tribal knowledge, or the invisible side effects of a framework.

The true cost of modification is not determined only by the amount of information required. It is determined by the cost of acquiring sufficient context with enough confidence that the context is complete. This includes the size of the context, the spread of the context, the traversal needed to reach it, and the reliability of the paths by which it can be found.

We call this operational burden **Context cost**: the cost of acquiring sufficient context for correct modification. Context cost includes not only the size of the required context, but also how that context is reached, how widely it is spread, and how reliably the modifier can know

that the acquired context is complete. Good design, then, is design that minimizes context cost.

4 The Context Minimization Principle

This intuition deserves a name. We call it the **Context Minimization Principle** (CMP). Made precise, it becomes a principle one can argue with:

For the modifications a system must realistically support, a design is better when the sufficient context required for correct modification is cheaper to acquire.

Four constraints do the real work.

Realistically support. CMP does not rank designs in the abstract. It evaluates them against the plausible stream of future modifications a system must support. Because that stream is uncertain, “realistically support” does not mean preparing for every imaginable future. It means reasoning from the product, domain, architecture, and organization about which changes are credible enough to shape the design. A plugin architecture is justified in a product that routinely adds third-party integrations; it is indefensible in a small internal CRUD tool.

Sufficient context. Sufficiency is an information condition relative to the correctness bar below: enough facts to eliminate the modifier’s uncertainty about whether the change is correct. CMP does not lower this threshold, it only asks the same sufficient set to be cheaper to acquire, and avoid unrelated context.

Correct modification. The bar is correctness: a change that is complete, preserves invariants, and produces the intended behavior.

Cheaper to acquire. Cheapness has two axes: *size* and *discoverability*. A sufficient set is cheaper when it contains fewer facts, and the facts are more directly reachable. A design that shrinks the set but leaves it hidden has not reduced cost; nor has a design that scatters the set across the codebase even if each fact is locally simple. Context size is theoretical; context acquisition cost is operational.

Taken together, these constraints turn CMP from a slogan into a comparative frame: scoped to realistic modifications, grounded in sufficiency, disciplined by correctness, and measured by context cost rather than by visible size alone.

5 Depth: Acquiring Behavioral Context

CMP names context but must also explain how context expands. The first recurring expansion is **depth**.

Depth is the cost of traversing dependencies until the behavior behind a focal point is sufficiently understood.

Consider a call to `calculateDiscount(order, customer)`. In one design, the rule lives in the function body and its tests. The modifier reads the function, checks the test, and stops. In another design, the same call dispatches through a `PricingService`, then a `DiscountPolicy` interface, then a runtime strategy factory, then a feature-flag provider, then a configuration file. The visible call site is the same, but the context cost is not.

Each indirection — a function, interface, inheritance edge, framework hook, configuration lookup, dependency injection container, or remote call — is a place where traversal might stop or continue. **Depth is not the enemy, failing to stop is.** Every useful abstraction adds some depth—that is the entire point of a boundary, in the tradition of information hiding and deep modules (Parnas 1972; Ousterhout 2018).

A boundary holds when its contract is sufficient for the modification at hand. A well-named function, precise type signature, stable interface, focused test, or explicit protocol can let the modifier reason from the surface without loading the implementation behind it. A boundary fails when the signature under-specifies behavior, runtime binding is opaque, or the contract is only knowable by reading the implementation. In that case, the design has added traversal without creating a reliable stopping point.

Depth also works in both directions. The caller should not have to traverse inward into the implementation to use a boundary correctly. The implementer should not have to traverse outward through all callers to know what the boundary promises. The same `DiscountPolicy` interface that lets a checkout-flow modification avoid reading the pricing implementation should also let the pricing implementation evolve without scanning every checkout call site.

Depth fails by exhaustion. The modifier eventually acquires the needed context, but only after opening more files, following more edges, holding more state in working memory, and resolving more indirection than the apparent surface suggested. The cost is paid by the modification already under way.

In CMP terms, good design does not eliminate depth. It makes each unit of depth earn its place by replacing larger or less reliable context acquisition with a boundary the modifier can trust.

6 Breadth: Acquiring Modification Closure

Understanding behavior is only half of correct modification. The modifier must also acquire the complete set of places that must be considered together for the change to be correct. This is the second recurring expansion: **breadth**.

Breadth is the cost of acquiring the modification closure: the set of artifacts that must be considered together because a realistic modification may require them to change together.

Suppose a discount rule is implemented independently in the checkout UI, the order service, the billing job, and the analytics pipeline. Each implementation may be locally clear. Depth may be shallow at each site. But a correct modification to the rule must consider all of them. The decision is scattered, and that scattering creates breadth.

The central point about breadth is that it is not textual similarity. Two loops can look identical while encoding unrelated decisions, and centralizing them invents a shared abstraction that does not exist. Conversely, a SQL schema, a frontend form, a backend validator, an analytics query, and a test fixture can encode the same domain decision without sharing a line of code. The test is not “do these lines look alike?” but:

Would a realistic modification require these places to change together?

If yes, they belong to the same modification closure. If the closure is scattered, the design must either co-locate it, index it, or accept omission risk.

This is where breadth differs from a mere count of locations. Some breadth is indexed. The participating artifacts may be scattered, but their relationship is discoverable through names, types, registries, architectural conventions, tests, generated code, event schemas, exhaustive matches, or dependency structure. Indexed breadth still costs something: the modifier must follow the index. But it is a bounded traversal problem.

Unindexed breadth is different. Artifacts must change together, but nothing in the system reliably reveals that relationship. The modifier must rely on memory, grep, code review luck, tribal knowledge, or full-codebase inspection. The practical failure mode is not extra reading; it is omission. A modifier updates three of four places, local tests pass, and the fourth surfaces later as a data inconsistency, reporting mismatch, or customer-visible bug.

Breadth therefore fails by oversight. Depth taxes the modification already being performed; breadth taxes the modifications the modifier does not know they must perform. The worst breadth is not necessarily the widest breadth. It is the unindexed breadth whose closure cannot be acquired with confidence.

This reframes a central design task. Good design does not only shrink modification closures by moving shared decisions into one place; this is the context-cost reading of DRY and bounded contexts (Hunt and Thomas 1999; Evans 2003). It also makes the remaining closure discoverable

when co-location is impossible or undesirable. Many architectural conventions, patterns, type-system features, registries, and test structures exist for exactly this purpose: they convert open-ended search into bounded closure acquisition.

7 Design as Context Transformation

Depth and breadth are independent axes, but most design moves affect both. A design move rarely eliminates context. It transforms context from one acquisition shape into another.

A design can reduce context, by moving a scattered decision into one named location. It can relocate context, by moving implementation details behind a boundary. It can index context, by creating a discoverable path among artifacts that still remain distributed. The payoff is the same in each case: sufficient context becomes cheaper to acquire for realistic modifications, thereby lowering context cost.

The trade is not symmetric. Depth fails by exhaustion, breadth fails by oversight. A change that took too long may still be correct. A change that missed a hidden participant in its modification closure is silently wrong. CMP therefore has a bias: it is often worth paying visible depth to remove or index real breadth. But that bias is not absolute. Visible depth paid for imaginary breadth is over-engineering; duplicated code with obvious, indexed, local breadth may be cheaper than abstraction.

The duplication-abstraction trade illustrates the point. This is the tension captured by warnings against premature or wrong abstractions (Metz 2016; Fowler 2018). Duplicate the discount rule across four call sites and any modification to the rule must acquire a closure of four places. If those four places are unindexed, the cost includes omission risk. The standard design move is to notice that the four sites encode one underlying decision, name it, implement it once, and place it behind a boundary such as `DiscountPolicy`. After the move, the closure collapses to one named decision plus its boundary. The cost is a new concept and one traversal hop. The benefit is that the future modifier no longer has to rediscover four scattered decisions.

But co-location is not the only transformation. Architecture can leave decisions distributed while making their locations predictable. A layered or hexagonal architecture adds depth — domain logic, ports, adapters, transport, persistence — but in exchange it turns open-ended search into directed lookup (Martin 2017; Cockburn 2005). A developer who knows where discount rules belong searches a smaller and more reliable space than one who must scan the system.

Patterns often work similarly (Gamma et al. 1994). A Strategy does not necessarily reduce the number of concrete policies; it creates an indexed policy axis. A State model does not eliminate state-specific behavior; it makes the state space explicit. An event schema does not remove consumers; it defines their relationship to the producer as a protocol (Hohpe and Woolf 2003). Strong type mechanisms do not remove cases; they make missing cases mechanically discoverable.

Both abstraction and indexing express the same underlying motion: design restructures context so that realistic future modifications can acquire sufficient context with less open-ended search, less hidden traversal, and less reliance on memory. The next question is when such restructuring is worth the cost. That is the proper subject of over-engineering.

8 The Over-Engineering Criterion

Every context transformation is a trade: a design move adds context cost — a new boundary, a new concept, a traversal hop, an index to learn — in exchange for removing, relocating, or indexing other context costs. Over-engineering is simply a bad trade: it costs more than it saves.

To judge whether a design decision is worthwhile, we must analyze it in two passes. First, we examine the design in a controlled environment where future requirements are known, reflecting the reality of pre-release development or highly constrained domains. Second, we confront the uncertainty of a living system, where design decisions behave less like accounting and more like financial speculation.

8.1 The Static Criterion: Known Modification Streams

Start with the simplest case: the relevant future modifications are known or highly constrained. We can hold one modification class fixed and ask whether a design move pays for itself.

In this static case, abstraction, layering, interfaces, registries, configuration, and patterns are neither inherently good nor bad. They are justified when the context cost they remove exceeds the context cost they add.

Every design move adds context cost in three recurring forms. They scale on different axes, so a move that looks cheap on one can still be expensive on another:

- **Boundary cost** — the interface every abstraction introduces (names, signatures, error semantics, invariants) that must be learned before its body can be ignored. The boundary is itself depth, paid in exchange for letting readers *stop* instead of traversing the hidden implementation. This is Ousterhout’s *deep* versus *shallow* module distinction in *A Philosophy of Software Design* (Ousterhout 2018): a deep module hides substantial complexity behind a small interface, while a shallow module imposes nearly as much cognitive load to learn its interface as reading the body directly — and therefore fails the trade because the depth added is barely converted into depth removed. Scales with the number of distinct modifiers and modifications that use the boundary.

- **Carrying depth** — depth introduced by a design decision that every unrelated modification must traverse before the anticipated benefit is realized. Not a one-time setup fee but a toll paid each time a developer walks past the boundary, including modifications the abstraction was never meant to help. Wrapping every database query in a multi-tenant routing layer to prepare for a future SaaS transition means a simple internal report today still has to traverse three layers of indirection. Scales with the unrelated modification stream.
- **Conceptual surface** — the vocabulary, invariants, and small mental model the move adds to the codebase as a whole, paid in naming, refactoring, code review, and onboarding even by code unrelated to the original problem. Scales with the concept’s reach, independent of any specific modification. *Lookup cost* — knowing that a registry or convention exists and how to use it — is the conceptual-surface tax specific to indexing moves.

A design move earns those costs only by removing larger ones elsewhere. The removed costs have matching named forms:

- **Depth skipped** — a reliable boundary lets the modifier read the contract instead of the implementation, saving whatever inference would otherwise reconstruct that contract from the body.
- **Closure collapsed** — an abstraction merges change-together sites into one named decision, shrinking breadth and the omission risk that came with unindexed scatter.
- **Closure indexed** — a registry, schema, exhaustive match, or generated artifact makes a distributed closure mechanically discoverable, converting open-ended search into bounded lookup without requiring co-location.
- **Implicit context checked** — tests, types, and language mechanisms move behavioral or structural assumptions out of memory into tool feedback.

The static judgment is therefore:

A design move is justified when the depth it skips, the closure it collapses or indexes, and the implicit context it makes checkable outweigh the boundary cost, carrying depth, and conceptual surface it adds.

8.2 The Temporal Criterion: Design Under Uncertainty

The static equation works perfectly when requirements are fixed, but real software development almost never enjoys this luxury. From the first line of code, design decisions are made under uncertainty. Every design move under uncertainty is therefore a wager: we pay a visible present cost, for an uncertain future benefit. Over-engineering is what happens when this wager is over-priced.

The clean way to reason is to compare two concrete worlds.

In the **build-now world**, the design move is made today. Suppose a team builds a multi-tenancy seam now: every query, cache key, background job, report, and session path is parameterized by tenant. The carrying cost includes the concepts, interfaces, routing rules, and traversal every unrelated modification must now load. The payoff arrives only if the SaaS future materializes along the anticipated axis.

In the **defer world**, the seam is not built today. Single-tenant assumptions remain local and direct. The present carrying cost is avoided. If SaaS arrives later, the retrofit cost is the future context cost of finding and realigning every place that assumed one tenant: queries, cache keys, URL routing, sessions, reports, tests, logs, permissions, and deployment assumptions.

The decision is not a precise calculation, but it is a structured comparison:

expected future context cost avoided versus present context cost paid.

YAGNI is the conclusion when the defer world’s future closure remains small, local, or discoverable (Beck 1999). Eager structuring is justified when deferral would leave a large or unindexed closure that future modifiers cannot reliably acquire.

Two risks appear only under uncertainty.

Wrong-shape risk belongs to the build-now world. The team may build a boundary or index for a future variation that arrives along a different axis. If a row-level `tenant_id` seam is built but the real requirement demands physical tenant isolation, the paid depth does not remove the future closure. Worse, the wrong abstraction becomes another structure future work must route around. Present context cost has been paid for a future that did not arrive.

Unindexed-closure risk belongs to the defer world. The unresolved decision does not disappear when deferred. If every query and cache key quietly assumes one tenant, the decision is already distributed. The danger is that nothing names it. The future retrofit cost is not merely large; it is hard to acquire because the closure is unindexed. The modifier does not know where the search ends.

These risks pull in opposite directions. Wrong-shape risk penalizes premature structuring. Unindexed-closure risk penalizes careless deferral. Practices such as YAGNI and the Rule of three can be read as calibration strategies (Beck 1999; Fowler 2018; Metz 2016): wait until enough concrete instances reveal the shape of the variation, but do not wait so long that an unnamed decision leaks across the system.

The central judgment resolves into four cases:

- **Justified eager structuring:** present context cost is smaller than the future closure cost the design move avoids or indexes.
- **YAGNI-correct deferral:** the future closure remains small, local, or discoverable enough that present structure would cost more than it saves.
- **Over-engineering:** present carrying depth, interface burden, conceptual surface, and wrong-shape risk exceed the future context cost actually avoided.

- **Under-engineering:** present simplicity leaves a realistic future modification closure unindexed, making sufficient context expensive or unreliable to acquire later.

Good design is neither maximally abstract nor maximally simple. It is the design whose present and future context costs are both made explicit, paying visible depth only where it prevents larger, less discoverable future cost.

8.3 Making Hidden Costs Nameable

The value of the over-engineering criterion is not that it produces exact numbers. Most probabilities are not measurable, and future retrofit costs are only approximated by judgment. Its value is that it makes the relevant costs visible and nameable.

A design debate often hardens into aesthetic camps: abstraction versus simplicity, architecture versus directness, YAGNI versus extensibility. CMP decomposes such disputes into two separable disagreements. One is about the design itself: what context cost does this move add, remove, relocate, or index? The other is about the future: which modification streams are credible enough to justify paying present cost?

The first disagreement can often converge through analysis: once teammates use the same names for what a move adds and removes, vocabulary like “over-abstracted” versus “cleanly factored” tends to dissolve into a shared cost ledger. The second remains uncertain, but it stops being a clash of taste and becomes an explicit bet on the modification stream. Many architectural disputes are therefore arguments about the future masquerading as arguments about the design.

9 Design Principles as Context Operators

Classical design principles are useful but incomplete because they rarely specify their own boundary conditions. They tell engineers to hide information, avoid duplication, reduce coupling, increase cohesion, depend on abstractions, test behavior, or separate policy from mechanism. But they do not always say when these moves are worth their cost, or how to decide when two principles point in opposite directions.

Under CMP, a design principle is a **context operator**: a rule for transforming the context cost of future modifications. Each principle proposes to move context from one acquisition shape to another — from implementation traversal into a responsibility contract, from scattered repetition into a named decision, from unindexed breadth into a discoverable closure, from unchecked assumptions into tests or types.

A principle can be evaluated by three questions:

1. What realistic modifications does it assume?

2. What context cost does it reduce — traversal depth, closure breadth, implicit context, or omission risk?
3. What context cost does it introduce — boundary cost, carrying depth, conceptual surface, or wrong-shape risk?

The failure condition follows from the trade. A principle backfires when the assumed modification stream does not materialize, when variation arrives along a different axis, or when the context it introduces costs more than the context cost it removes.

9.1 Boundary Principles: Responsibility Contracts as Traversal Stops

Boundary principles control depth by stopping context traversal. A boundary is not created merely by hiding code behind a module, class, function, or interface. It is created when a design abstracts a responsibility, expresses that responsibility as a contract, and allows modifiers to reason from that contract without traversing into the implementation behind it.

A nominal boundary hides code. A semantic boundary hides context.

Information hiding is therefore the effect of a successful boundary, not its root mechanism (Parnas 1972; Ousterhout 2018). Implementation details can be excluded from required context only when they are details of a well-defined responsibility. If the responsibility is vague, unstable, or under-specified, the modifier must still cross the boundary to recover what the contract failed to express. The design has added depth without creating a reliable traversal stop.

A boundary also isolates in both directions. Callers can rely on the contract without loading implementation decisions; implementers can change the implementation without enumerating every caller, as long as the contract remains satisfied. A successful boundary separates two local contexts: the caller does not import implementation details, and the implementer does not import caller-specific assumptions.

Several SOLID principles fit this boundary vocabulary (Martin 2002). Liskov Substitution says that a subtype boundary holds when clients can rely on the supertype contract without loading subtype-specific behavior. Interface Segregation says that clients should receive only the context required by their modification class. Open/Closed says that a boundary is worth paying for when future modifications are expected to arrive along the variation axis represented by the contract; if variation arrives along a different axis, the boundary becomes wrong-shape depth.

Dependency Inversion is a central boundary mechanism because it makes context requirements explicit at the client boundary. Its value is not the assembly flexibility of swapping one provider for another. Its value is that the client's required context is declared in its own contract: the collaborator boundary is shaped by what the client needs to fulfill its responsibility, not by what a concrete provider happens to expose.

The shift is easiest to see by analogy with product design. A supplier-designed product tries to put every capability the supplier *can* offer onto the shelf; a customer-designed product only sells what the customer is willing to pay for. Abstractions split the same way: a provider-shaped abstraction mirrors one implementation’s public surface, while a client-shaped abstraction sells only the capability the client is willing to “pay for” in context — what it must load to fulfill its responsibility. This is the precondition that LSP and ISP quietly assume. Interface Segregation asks that clients receive only the context they require; Liskov Substitution asks that any conforming provider be substitutable without forcing the client to load provider-specific behavior. A provider-shaped contract makes both harder to hold, because its surface tends to carry methods, error modes, and assumptions some client never asked for. When the dependency is inverted and the contract is authored from the client’s side, the boundaries LSP and ISP demand tend to emerge as a natural consequence rather than as a design that must be policed.

Boundary principles therefore do not prescribe interfaces, layers, or indirection as ends in themselves. They prescribe responsibility contracts strong enough to exclude implementation details, collaborator construction, provider-specific behavior, and caller-specific assumptions from the required context of realistic modifications. When that condition holds, a boundary reduces depth. When it does not, the boundary is only extra context the modifier must acquire.

9.2 Locality Principles: Making Modification Closure Reachable

Locality principles control breadth by reducing the cost of acquiring modification closure. Traditional formulations capture a real design pressure, but their vocabulary is often under-specified. DRY says not to duplicate knowledge (Hunt and Thomas 1999). SRP says a module should have one reason to change (Martin 2002). Cohesion says related things should stay together (Yourdon and Constantine 1979). The shared intuition is clear, but the key terms are not: what counts as the same knowledge, one reason, related things, or change together?

CMP gives this intuition a more operational object: **modification closure**. For any concrete modification a developer is asked to perform — fixing this bug, adding this field, changing this rule — there is a definite, enumerable set of artifacts that must be edited together for the change to be correct. That set is the modification closure of that change. It is not a hypothetical category like “one reason to change”; it is a concrete property of each actual modification. Every change a system realistically supports has such a closure, and breadth is the cost of acquiring it. Locality principles reduce breadth by making each closure smaller when possible and reachable when necessary.

This idea is not theoretical; experienced engineers detect it through an automatic intuition. While writing a feature, you find yourself editing several places — a schema column, a validator, a UI field, a job — and notice that nothing in the code records the fact that these places belong together. Their relationship lives only in the decision you happen to be making right now.

That observation is what fires a code-smell alarm; it is not a deliberate check for omissions, it is a reflex an experienced engineer cannot help running on whatever structure they are editing. This is the sense of modification closure, a real thing lives in your head.

By our definition, only what the system’s explicit artifacts record counts. A closure that lives only in someone’s head is considered unreachable from the artifact system. This gives locality an operational test. Pick any *legitimate modification entry point* — any place from which a realistic modification would naturally begin: a failing test, a changed field, an API endpoint, a schema column, a UI option, an event payload, a piece of business logic. From that entry point, ask whether the rest of the closure can be reached through mechanical or structural paths in the code itself.

Locality is the property that, from any legitimate modification entry point, the complete modification closure can be reached through paths recorded in the system’s explicit artifacts.

Those paths take many forms — names, types, registries, generated artifacts, architectural conventions, test matrices, event schemas, exhaustive matches, dependency structure. Their common purpose is to convert open-ended search into bounded closure acquisition. Distributed context is not inherently bad; it becomes dangerous when the relationship among closure members lives only in someone’s memory. Co-location is the trivial case: when a decision lives in one place, every entry point reaches the entire closure in zero steps.

This operational test reorganizes the classical locality principles. They are not a list of independent rules; they are the same reachability judgment seen from two complementary directions.

DRY asks whether multiple artifacts encode the same decision. When they do, every realistic modification of that decision will touch all of them, so locality requires every such artifact to be reachable from every other. The cost of violating DRY is not duplication itself, but the cost and risk of acquiring all unindexed representations of the same decision. The optimal solution is collapse — one owned place, a trivial size-1 closure. When collapse is impossible or undesirable, an explicit index suffices. DRY fails when textual similarity is mistaken for decision identity: centralizing code that merely looks alike creates wrong-shape depth without reducing real breadth.

SRP and cohesion look at the same reachability picture from the opposite direction. Where DRY starts at an entry point and asks whether it reaches *the rest of* its closure, SRP and cohesion start at a module and ask whether the closures sitting inside it overlap with closures they do not belong to. A module is cohesive when the artifacts it contains serve a coherent family of modification closures; it loses cohesion when several unrelated closures share the same module. “One reason to change” is this non-overlap test in disguise, and its failure mode is concrete: a modifier sets out to change one closure but cannot make the edit without disturbing data structures, helpers, or invariants that belong to neighboring ones. Collateral edits creep

in, scope expands beyond the original closure, and a change that should have been local turns into a refactor of unrelated code.

Engineers have accumulated a toolbox for implementing locality when collapse is unavailable — type-system features, structural design patterns, runtime registries, schema conventions. These tools are not additional principles; they are recurring, named *index templates* that connect an entry point to the rest of its closure. Each can be read with the same template: *entry point is X; closure is Y; the index that connects them is Z*.

- **Algebraic data type with exhaustive match** — entry point: any operation over the type (Pierce 2002). Closure: every variant and every handler that consumes one. Index: the sum type itself, together with the compiler’s exhaustiveness check. Adding a variant turns closure acquisition into a list of compile errors: the modifier is mechanically marched through every site that must change, with no opportunity to forget one.
- **Strategy** — entry point: a policy call site (Gamma et al. 1994). Closure: every concrete policy implementation. Index: a strategy interface plus its dispatch. Implementations are reachable by following the interface’s type hierarchy, and the contract guarantees that each one exposes the same operation shape — so the closure is enumerable from the type even though discovery itself is a structural walk through the codebase rather than a compile-time check.
- **Plugin registry** — entry point: a plugin’s own registration declaration. Closure: every other registered plugin and the dispatcher that consults them. Index: a registry keyed on a stable identifier, looked up at runtime. The registry is ordinary code, but it is *one* discoverable place where every member of the closure announces itself; a modifier adding a plugin reaches the rest simply by reading what is already registered there.
- **Event schema with documented ownership** — entry point: the event definition (Hohpe and Woolf 2003). Closure: the producer, every consumer, and the protocol that binds them (ordering, retries, delivery semantics, compatibility). Index: the schema together with an ownership record that names the producer and each consumer. The schema is structural, but the consumer set lives in the ownership record — a social index, strong enough to keep consumers reachable from the producer side as long as the record is maintained.

The four examples also reveal another factor: indexes vary in strength, and that strength determines how reliably they can substitute for co-location.

- *Mechanical* — exhaustive matches over a sum type, generated code, a typed registry, a schema-driven artifact. The compiler or build tool refuses to proceed when the index is broken, so omission is mechanically prevented.
- *Structural* — naming conventions, directory layout, architectural layers, dependency edges. The index is readable in the code but not enforced; a careful modifier can follow it, a careless one can ignore it.
- *Social* — documentation, code comments, team conventions, tribal knowledge. The index exists only in memory and writing, and degrades silently as ownership turns over.

Mechanical indexes cost the most up front and pay the most by converting breadth failures into compile-time errors; social indexes are cheapest to introduce and the first to fail under personnel turnover. Most real systems live in the structural middle, and choosing the right strength for a given closure is itself a design decision.

9.3 Architecture: System-Level Context Routing

Architecture applies context operators at system scale. Boundary principles make local traversal stoppable; locality principles make modification closures reachable from concrete entry points. Architecture adds a larger function: it makes decision placement predictable across the system. It tells modifiers where a decision should live, which acquisition paths are legitimate, and which parts of the system should not need to be searched.

In CMP terms, architecture is **context routing**. Without architecture, acquiring context often degenerates into full-codebase search. With credible architecture, a modifier routes the search through bounded contexts, layers, ports, adapters, components, schemas, or other explicit structures. The architecture earns its carrying depth when these routes reliably reduce open-ended search.

Different architectural styles define different routing topologies. **DDD bounded contexts** route domain language and model changes within explicit semantic boundaries, so a modifier does not have to load a universal enterprise model for every local domain change (Evans 2003). **Clean Architecture** routes policy inward and mechanisms outward, so domain modifications can avoid framework, database, UI, and transport context unless those boundaries are the subject of the change (Martin 2017). **CQRS** routes mutation correctness and read-model optimization into separate acquisition paths, so command invariants and query projections can evolve under different context requirements (Fowler 2011). **Entity-Component-System** routes state through components and behavior through systems, making feature changes acquire context along component and system axes rather than through object hierarchies.

These styles differ in vocabulary and structure, but under CMP they make the same kind of promise: a modifier should not need to search everywhere. The architecture predicts where certain decisions belong. Domain vocabulary belongs inside a bounded context. Read models belong on the query side. Component state belongs in components. When these placement rules hold, architecture reduces context cost not necessarily by reducing the amount of code, but by making the acquisition path bounded and trustworthy.

This also explains why small architectural violations can be disproportionately damaging. A domain rule slipped into a controller, a persistence assumption leaking into a domain model, a cross-context shortcut tucked into shared code, or a command-side invariant duplicated in a query projection may each look harmless locally — the immediate change is even faster. But every such violation weakens the placement rule future modifiers rely on to know where such decisions should be found, and once that trust is gone, the architecture stops narrowing the search space: modifiers first look where the architecture says the decision should be, then

search the whole system anyway. Architectural decay, in CMP terms, is therefore not impurity as an aesthetic defect; it is the loss of reliable context routing. The system still has its layers, boundaries, conventions, and vocabulary, but they no longer predict where decisions live — the carrying depth of architecture is paid in full while the search-reduction benefit silently disappears. The map remains visible, but it no longer routes context acquisition.

9.4 Tests: Making Required Context Executable

Engineers often treat testability as a sign of good design — code that resists testing tends to feel coupled, hidden, stateful, or tangled (Beck 2002). CMP explains why this intuition is reliable. A test makes part of the required context of a modification executable: it records a behavioral fact the system must keep satisfying and rejects modifications that violate it, sparing the modifier from carrying every behavioral assumption in memory.

Testability follows from the same frame. To test a behavior, the modifier must acquire and control the context needed to exercise, observe, and verify it: an entry point, input state, dependencies, side effects, and a comparable outcome. When this context is cheap to acquire, the code is testable. When testing requires booting the whole application, preparing hidden global state, coordinating real infrastructure, or inspecting incidental internal steps, the design has made required context expensive.

This is why testability is a design signal. Code that is hard to test often has unclear boundaries, hidden dependencies, uncontrolled side effects, or behavior whose observable result is far from its cause. These are not testing problems alone. They are context problems exposed by testing. A testable design makes the relevant behavior reachable through a stable entry point, lets dependencies be controlled, keeps state local enough to construct, isolates side effects enough to observe, and exposes outcomes that can be compared against expectations.

Tests themselves can add context cost when they encode the wrong thing. Over-specified tests — mirroring private implementation, asserting internal call sequences, or mocking every helper — create parallel breadth: future modifiers must update the code and its test representation in lockstep even when externally relevant behavior has not changed. Under-specified suites have the opposite failure: they pass without checking the constraints that matter and produce false confidence. Flaky tests fail a third way, rejecting modifications unreliably and forcing modifiers to spend context on the test artifact rather than on the behavior it was meant to protect.

Testability is therefore not merely a convenience for writing tests. It is evidence that required context is observable, controllable, and cheap to acquire. Good tests lower the cost of confidence in a modification. Bad tests either fail to check the context that matters or add new context that future modifiers must carry.

9.5 From Principles to Context Operators

The point of this section is not to replace classical design principles with another principle. It is to make their trade-offs explicit. Boundary principles reduce traversal depth when their contracts are strong enough to let modifiers stop. Locality principles reduce breadth when modification closures are collapsed or reachable. Architecture reduces search cost when decision placement remains predictable. Tests reduce confidence cost when required context is made executable.

Once principles are read as context operators, their conflicts no longer require a hierarchy of rules. DRY does not always beat simplicity. Architecture does not always beat directness. Tests do not always reduce cost. Each move must be judged by the same question: for the realistic modifications the system must support, does it make sufficient context cheaper to acquire than the context cost it adds?

This is the role CMP gives to design principles. They remain useful, but they become conditional instruments rather than absolute commands. Their value depends on the modification stream they assume, the context cost they reduce, and the new context cost they introduce. The next section applies the same lens to programming languages, where some forms of context are not merely organized or tested, but mechanically enforced.

10 Programming Languages as Universal Mechanical Indexes

Programming languages occupy a peculiar position among context operators (Pierce 2002). Boundary principles, locality patterns, architecture, and tests all operate at the level of a particular module, a particular registry, or a particular convention; each indexes a specific closure but cannot guarantee anything beyond its own scope. Programming languages, by contrast, apply their context-control mechanisms uniformly across the entire codebase and enforce them mechanically through the compiler or runtime. The combination of universal reach and mechanical enforcement is what makes language design itself a CMP-level concern rather than a stylistic one.

Programming languages are the only context operator that is both *universal* — applied to every artifact in the codebase — and *mechanical* — its violations are detected by tools rather than by readers.

Read against the index strength axis introduced for locality, language mechanisms occupy its extreme: stronger than any structural convention because they cannot be ignored, broader than any single pattern because they apply everywhere. Choosing a language, or choosing which of its mechanisms to use, is therefore choosing which kinds of context will be mechanically indexed for every modifier who ever touches the codebase.

10.1 The Core Mechanism: Converting Breadth Failures into Depth Failures

The unique value of language mechanisms is not that they reduce context. Most of them in fact *add* visible context to the codebase: a type signature, a lifetime annotation, an effect tag, a visibility modifier all impose additional surface that every reader must learn. Their value lies in how they change the *failure mode* of modifications.

Recall the asymmetry between depth and breadth. Depth fails by exhaustion: the modifier eventually acquires the needed context, but only after more traversal than the surface suggested. Breadth fails by oversight: the modifier silently misses a member of the modification closure, and the system fails later in a context the modifier no longer controls. The two failures are not comparable. A slow change is still correct; an incomplete change is silently wrong.

Language mechanisms exploit this asymmetry directly. When a sum-typed variant gains a new case, the compiler enumerates every site that consumes it. When a nullable boundary is introduced, the compiler enumerates every site that ignored absence. When a function's effect tag changes, the compiler enumerates every caller that assumed the old profile. None of these mechanisms reduce the number of locations that must change. They make the modifier walk through them mechanically rather than from memory.

A language mechanism converts a previously unindexed closure into a list of compiler errors. It does not shrink breadth; it turns breadth-fails-by-oversight into depth-fails-by-exhaustion.

This is the sharpest claim of this section. It explains why even minimal type systems have outsized leverage compared to documentation or convention: the closure that would have failed silently now fails loudly. It also explains why language mechanisms have a ceiling — they can only redirect closures whose shape they can statically describe.

10.2 Language Mechanisms in the Cost Ledger

Before turning to long-running controversies, it helps to read the main language mechanisms through the cost ledger established earlier. Each one is a trade: it adds particular forms of boundary cost, carrying depth, or conceptual surface in exchange for particular forms of relief — depth skipped at boundaries, closures mechanically indexed, implicit context made checkable. Naming both sides removes the temptation to read any feature as straightforwardly good or bad, and gives the next section a shared vocabulary.

Static types pay boundary cost at every signature and add a small but pervasive conceptual surface across the codebase, in exchange for mechanically indexing shape closures and letting callers reason from signatures rather than implementations (Pierce 2002). The depth they skip scales with how often a value's shape would otherwise have to be reconstructed from scattered call sites.

Nullability and option types pay carrying depth wherever absence is structurally possible: every consumer must encode the absent case, even where it is rare in practice. In exchange, the absence closure — the set of sites that must agree on what missing means — is mechanically indexed, and the implicit assumption “this is never null here” becomes a checked branch.

Algebraic data types with exhaustive matching pay the cost of enumerating cases at every consumption site, including consumers that only care about one variant (Pierce 2002). In exchange, the variant closure becomes the cleanest mechanical index any language can offer: adding a case stops being a search problem and becomes a list of compile errors.

Module systems and visibility pay the boundary cost of declaring an exported contract and the conceptual surface of public-versus-private discipline. In exchange, callers depend on contracts rather than implementations: implementation details are excluded from required context, and the responsibility-contract boundaries described earlier become enforceable instead of merely conventional.

Immutability pays the carrying depth of functional update — every “small in-place change” becomes a value-construction operation — in exchange for collapsing the state-history closure. A value that cannot change does not require the modifier to acquire its prior writes.

Explicit error and effect mechanisms pay carrying depth along the entire call chain (Nielson et al. 1999): every caller must declare or propagate the effect. In exchange, the effect closure — the set of sites that must agree on what may fail or what side effects may occur — is mechanically indexed at function boundaries.

Ownership, borrowing, and lifetime systems pay the heaviest carrying depth and conceptual surface of any mainstream language mechanism (Klabnik and Nichols 2023): every signature and every binding declares its aliasing and lifetime story. In exchange, the aliasing closure — historically one of the most unindexed and silently-failing closures in software — becomes a compiler-checked structure.

Two observations carry into the next section. First, the **payoff scales with the size and density of the corresponding closure**, while the **cost is paid uniformly across the codebase**. A mechanism that mechanically indexes a closure that is small or rare in the system’s actual modification stream is paying universal carrying depth for an empty index. Second, the same mechanism may be a clear gain in one system and a clear loss in another, because the modification streams differ. The cost ledger does not change; the closures it operates on do.

10.3 Long-Running Controversies as Hidden-Cost Disagreements

The most durable disagreements in language design — static versus dynamic typing, ownership systems versus garbage collection, effect systems versus unconstrained side effects — share a recognizable shape. Each side correctly observes a real context cost. Each side’s everyday systems make a different cost the dominant one. The disagreement is therefore not a clash of

values but an empirical disagreement about which closures dominate the modification stream a system actually faces.

This is the same shape that surfaces inside a single engineer’s design choices when over-engineering is named, applied between communities. CMP cannot adjudicate the empirical question, but it can name what each side is observing and what each side is structurally positioned to ignore.

Static versus dynamic typing. Proponents of static typing are correctly observing the omission risk of unindexed shape and absence closures: a renamed field, a new variant, a forgotten nullable boundary will fail silently across dozens of call sites unless the closure is mechanically indexed. Proponents of dynamic typing are correctly observing the boundary cost, carrying depth, and conceptual surface that types impose on every signature in the codebase — a tax paid even by modifications the type system never had to help. The blindness is symmetric. The static camp tends to discount that, in exploratory code, prototypes, scripts, or systems whose closures have not yet stabilized, the type tax is wrong-shape depth: it indexes a closure whose final shape is still being discovered. The dynamic camp tends to discount that, in systems where closures have stabilized and grown wide, omission risk becomes silent and expensive — its absence from their daily experience is partly a function of working in systems where closures happen to be small, or where tests and tribal knowledge act as structural or social indexes that mask the problem.

Read this way, the long history of gradual typing stops looking like a pragmatic compromise between camps (Siek and Taha 2006). It is the direct CMP-shaped solution: pay boundary cost only at sites where the closure has stabilized enough to deserve a mechanical index, and defer the cost everywhere else. Gradual typing operationalizes the temporal criterion of over-engineering at the granularity of individual signatures.

Ownership systems versus garbage collection. Proponents of ownership are correctly observing that aliasing closures, when they exist, are among the worst kinds of unindexed breadth — data races, use-after-free, and double-free are silent failures whose required context spans the entire reachable lifetime of a value. A mechanical index here pays back catastrophically large omission risk. Proponents of garbage collection and simpler memory models are correctly observing that ownership annotations are universal carrying depth: every function, including the large majority whose data flow is a straightforward request-response cycle with no real aliasing closure, pays the cost of lifetime and borrowing discipline. The blindness is again symmetric. Ownership advocates tend to discount how often, in ordinary application code, the aliasing closure is essentially empty — there is no shared mutable state worth indexing, and the language is paying carrying depth for an absent index. The opposing camp tends to discount how silently and expensively aliasing fails in systems software, concurrent runtimes, embedded code, or any setting where multiple actors share long-lived mutable state. The recurring debate over what ownership-based languages are “good for” is, under CMP, a debate about which systems have aliasing closures dense enough to justify a universal mechanical index for them.

Effect systems versus unconstrained side effects. Proponents of effect systems are correctly observing that effect closures fail silently when unindexed: a blocking call inside a hot path, hidden IO inside a function read as pure, an unawaited async boundary — each is an omission a reader cannot reliably catch. Proponents of pragmatic, untracked effects are correctly observing that when nearly all code in a system performs the same effects — almost everything does IO, almost everything is async — effect annotations are carrying depth with no corresponding depth skipped, because the index does not differentiate anything. The blindness here is shaped slightly differently. Effect-system advocates tend to discount that the value of an effect index is proportional to *effect diversity*: in a uniformly-effectful codebase, the annotation is a tax without information. Pragmatic advocates tend to discount that in the rare locations where effect boundaries do matter — a synchronous path that must not block, a “view” function that must not mutate — the failure is silent and the recovery is expensive.

In all three controversies the pattern is the same: each side has accurate experience of one column of the cost ledger and structural blindness to the other, and what looks like a philosophical disagreement is more precisely a disagreement about the modification streams of the systems each camp typically inhabits.

10.4 What Language Mechanisms Cannot Index

The reach of language mechanisms is real but bounded. They can mechanically index the closures whose membership is statically derivable from the program’s syntactic and type-level structure: shape, absence, variant, visibility, aliasing, effect under some discipline. They cannot index closures whose membership depends on facts the language does not see — domain semantics, cross-service protocols, deployment topology, ownership across organizational boundaries, product intent, or the change-together relationship between a SQL schema and an analytics dashboard. These closures remain the domain of the operators developed earlier: responsibility-contract boundaries, locality patterns, architectural placement rules, and tests. None of those is universal-and-mechanical in the language sense, but each can index closures that no type system reaches.

A useful way to read the language layer is therefore as the **floor of mechanical context indexing**: it handles the closures every modifier will encounter regardless of system, and it does so at a strength no convention can match. The remaining closures — usually the more domain-specific and higher-leverage ones — are handed back to the rest of the design vocabulary.

This is also the reason language mechanisms deserve special attention in what follows. AI agents, more than human engineers, depend on context that is explicitly recorded and mechanically checkable. Closures indexed by the language are closures an agent can follow without inference; closures indexed only socially or structurally are closures an agent must reconstruct from weaker signals. The language floor is therefore not just a foundation for human modifiers; it is

the part of the context infrastructure most directly legible to the agents that increasingly share the work.

11 AI-Assisted Engineering and Observable Context

AI-assisted engineering does not change CMP; it makes CMP easier to observe. An AI coding agent is another modifier of the codebase, as recent software-engineering agent benchmarks and interfaces make concrete (Jimenez et al. 2024; Yang et al. 2024). To make a correct change, it must acquire sufficient context, locate the relevant decisions, respect existing boundaries, and verify that the modification preserves intended behavior.

The difference is that agents depend more directly on explicit artifacts. A senior developer may rely on memory, architectural intuition, team convention, and historical background. An agent must recover more of that knowledge from code, tests, documentation, dependency structure, and tool feedback. A design that places invariants in types, localizes decisions in modules, expresses behavior in tests, and gives concepts predictable architectural locations is therefore more legible to both humans and machines.

AI-assisted engineering also exposes traces of context acquisition: retrieved files, search attempts, dependency traversal, prompt size, tool calls, localization failures, test failures, retries, patch size, and review corrections. These traces are not the context cost itself, but they are observable proxies for the process of acquiring context.

Depth appears when an agent must follow a long chain of calls, interfaces, configuration, framework hooks, runtime bindings, or remote services before it can understand behavior. Breadth appears when a change requires global search to acquire every schema, validator, UI, test, job, report, or event consumer that encodes the same decision. Missed edits, failed tests, repeated localization attempts, and oversized retrieval sets are symptoms that required context was scattered, implicit, or insufficiently indexed.

This does not mean humans and agents process context identically. Humans use tacit memory, visual scanning, social knowledge, and domain experience. Agents rely more heavily on explicit, retrievable artifacts. But the overlap is exactly what CMP predicts: both benefit from reliable boundaries, localized decisions, explicit contracts, coherent tests, predictable architecture, and strong language mechanisms.

AI-assisted engineering therefore gives CMP a research opportunity. It does not prove that context acquisition cost is fully measurable, but it exposes part of the acquisition process that was previously hidden inside human cognition. The practical lesson is immediate: designing for modification increasingly means designing artifacts that make context discoverable, bounded, and checkable — not only for experienced maintainers, but also for AI agents participating in future software work.

12 Scope and Limits

CMP makes strong claims: that software design is for modification, that context acquisition is the currency of design, and that classical principles can be reframed as context operators. These claims do real work, but they require limits.

12.1 The Scope of CMP's Verdict

Throughout this paper, phrases such as good design and better design should be read with a qualifier: better with respect to modifiability. A design that minimizes context acquisition cost for realistic modifications can still be incorrect, slow, insecure, unreliable, unusable, non-compliant, uneconomic, or shipped too late. CMP does not adjudicate those dimensions.

The principle gives a frame for one important trade-off space. It does not claim to integrate every trade-off that matters in software engineering.

12.2 Where CMP Applies

CMP is modification-relative. Its judgments presuppose a credible expectation about which modifications a system will realistically face. Where no meaningful modification stream exists, CMP has little to say.

The clearest cases are one-off scripts, exploratory prototypes, throwaway demos, and teaching examples. In such systems, the cost of acquiring context for future modification is paid at most once or never. Design choices may instead be governed by speed, exposition, experimentation, or single-use clarity.

Even in long-lived systems, the realistic modification stream is an empirical bet. Reasonable engineers can disagree about which futures are credible. CMP clarifies what each side is betting on, but it cannot make the future certain.

12.3 The Limits of the Depth/Breadth Model

Depth and breadth are tractable abstractions, not an exhaustive taxonomy of context acquisition. Temporal context, social context, protocol context, operational context, and organizational ownership may all impose costs that do not fit neatly into the two axes.

A second limit concerns boundary reliability. CMP says that a boundary matters when it stops traversal, but determining whether a boundary is reliable is itself contextual. It can depend on contracts, tests, names, conventions, types, reader familiarity, and organizational trust. CMP names the property that matters; it does not provide a mechanical procedure for evaluating it.

A third limit concerns closure identity. Breadth depends on determining whether artifacts belong to the same change-together relationship. This is not reducible to textual similarity and cannot always be derived automatically. It remains an engineering judgment grounded in domain semantics and realistic modification expectations.

12.4 Measurement and Operationalization

This paper proposes CMP as an analytical lens, not a completed metric. Treating context acquisition cost as a comparable scalar would require resolving several open problems:

- **Decision identity:** when do artifacts encode the same underlying decision?
- **Boundary reliability:** when does an interface, type, module, or test function as a true traversal stop rather than nominal indirection?
- **Closure indexing:** how can we detect whether a modification closure is discoverable or unindexed?
- **Modification distributions:** what set of realistic modifications should a design be evaluated against?
- **Gaming:** how can a metric avoid rewarding designs that hide necessary context behind opaque surfaces?
- **Socio-technical variance:** how do team memory, ownership, review norms, and operational experience affect actual acquisition cost?

The same caveat applies to AI-agent traces. They are useful observation windows onto context acquisition, but they are not direct measurements of context cost. A trace reflects the agent’s capabilities, prompt design, retrieval strategy, and toolchain as well as the codebase.

Taken together, these limits constrain CMP’s claim. It is a comparative frame for reasoning about modifiability trade-offs. Making the principle measurable remains future work.

13 Conclusion

Software is built to be modified. Correct modification requires sufficient context. The design question that follows is not merely how much context exists, but how costly it is to acquire enough context to change the system correctly.

CMP’s central refinement is therefore operational: good design minimizes the cost of acquiring sufficient context for realistic modifications. Context expands mainly through depth and breadth. Depth is the traversal cost of understanding behavior behind a focal point. Breadth is the closure cost of acquiring all locations that must change together. Design improves modifiability by reducing, relocating, indexing, or checking this context.

This reframing makes classical design principles comparable. Boundaries create responsibility contracts that stop traversal. Locality principles make change-together relationships co-located

or indexed. Architecture turns open-ended search into directed lookup. Patterns give recurring modification closures recognizable topology. Tests and language mechanisms make required context observable and checkable. Over-engineering becomes a bad acquisition trade: paying present context cost for future savings that do not arrive, arrive along the wrong axis, or would have remained cheap to acquire without the structure.

CMP is not a new design technique. It is a shared currency in which familiar techniques can be compared. When an architectural disagreement is expressed in that currency, it decomposes into two questions: what acquisition cost does this design move add, remove, relocate, or index; and what future modifications are credible enough to justify paying that cost?

Two research directions follow. The first treats CMP as a reasoning style for AI coding agents: agents that reason about depth, breadth, closure indexing, and over-engineering may make better design choices. The second develops quantitative indicators of context acquisition cost as proxies for design quality. Agent traces, static structure, test failures, retrieval patterns, and modification histories may all expose pieces of the acquisition process.

Both directions remain open. This paper proposes CMP as the analytical groundwork: software design is for modification, and the modifiability of a design is determined by how cheaply future modifiers can acquire the sufficient context needed to change it correctly.

References

- Beck, Kent. 1999. *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Beck, Kent. 2002. *Test-Driven Development: By Example*. Addison-Wesley Professional.
- Cockburn, Alistair. 2005. *Hexagonal Architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>.
- Evans, Eric. 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Fowler, Martin. 2011. *CQRS*. <https://martinfowler.com/bliki/CQRS.html>.
- Fowler, Martin. 2018. *Refactoring: Improving the Design of Existing Code*. 2nd ed. Addison-Wesley Professional.
- Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- Hohpe, Gregor, and Bobby Woolf. 2003. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional.

- Hunt, Andrew, and David Thomas. 1999. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley.
- Jimenez, Carlos E., John Yang, Alexander Wettig, et al. 2024. “SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?” *Proceedings of the International Conference on Learning Representations*. <https://arxiv.org/abs/2310.06770>.
- Klabnik, Steve, and Carol Nichols. 2023. *The Rust Programming Language*. 2nd ed. No Starch Press.
- Lehman, M. M. 1980. “Programs, Life Cycles, and Laws of Software Evolution.” *Proceedings of the IEEE* 68 (9): 1060–76. <https://doi.org/10.1109/PROC.1980.11805>.
- Martin, Robert C. 2002. *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.
- Martin, Robert C. 2017. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall.
- Metz, Sandi. 2016. *The Wrong Abstraction*. <https://sandimetz.com/blog/2016/1/20/the-wrong-abstraction>.
- Nielson, Flemming, Hanne Riis Nielson, and Chris Hankin. 1999. *Principles of Program Analysis*. Springer.
- Ousterhout, John. 2018. *A Philosophy of Software Design*. Yaknyam Press.
- Parnas, David L. 1972. “On the Criteria to Be Used in Decomposing Systems into Modules.” *Communications of the ACM* 15 (12): 1053–58. <https://doi.org/10.1145/361598.361623>.
- Pierce, Benjamin C. 2002. *Types and Programming Languages*. MIT Press.
- Siek, Jeremy G., and Walid Taha. 2006. “Gradual Typing for Functional Languages.” *Scheme and Functional Programming Workshop*, 81–92.
- Yang, John, Carlos E. Jimenez, Alexander Wettig, et al. 2024. “SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering.” *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2405.15793>.
- Yourdon, Edward, and Larry L. Constantine. 1979. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Prentice Hall.